

Day 39 — Line-by-Line Syntax Explanation

day39_softmax_minibatch.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 6, Day 4 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula (softmax, categorical cross-entropy, the softmax+cross-entropy gradient), a purple **MATH** box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green **SYSTEM DIAGRAM** shows how that math and code fit together as a working system — including the extended training work cycle, and the script's own real output graphs.

MATH box legend: $\times$ = matrix multiply / times $W1, b1$ = weights/bias of layer 1, etc. X^T = transpose of X
 $\text{mean}[...] = \text{average over all examples}$ m = number of examples in this step K = number of classes

Segment 0 — Imports and Global Setup (lines 31–37)

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

np.random.seed(42)
```

Identical setup pattern to Days 36-38: **np** for all array math, **matplotlib.use("Agg")** before importing **pyplot** so charts render straight to PNG files instead of a GUI window, **make_blobs** imported directly to build this lesson's 3-class dataset (in place of the earlier days' **make_moons**), and **np.random.seed(42)** fixing every subsequent "random" NumPy call for reproducibility.

Segment 1 — Softmax (lines 44–65)

MATH

$z_shifted = z - \max(z, \text{per row})$ (numerically safe, changes nothing mathematically)
 $\text{softmax}(z) = \exp(z_shifted) / \sum(\exp(z_shifted), \text{per row})$

```
def softmax(z):  
    z_shifted = z - z.max(axis=1, keepdims=True)  
    exp_z = np.exp(z_shifted)  
    return exp_z / exp_z.sum(axis=1, keepdims=True)
```

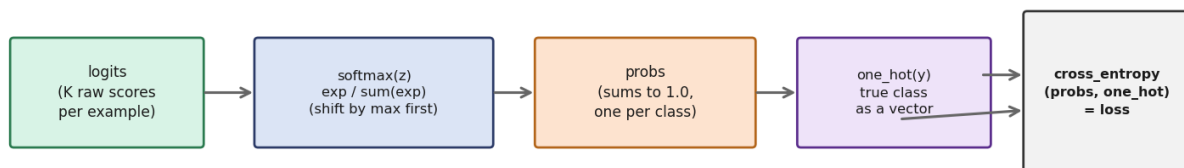
z is shaped (m, K) — one row of K logits per example. **z.max(axis=1, keepdims=True)** computes each ROW's maximum (**axis=1** reduces across columns, leaving one value per row); **keepdims=True** keeps the result shaped (m, 1) instead of flattening to (m,), which is what allows the subtraction on the next line to **broadcast** correctly against the full (m, K) array. The same **axis=1, keepdims=True** pattern repeats in the final **.sum(...)**, dividing each row by ITS OWN total rather than one shared total across the whole batch.

```
def demo_softmax():  
    logits = np.array([[2.0, 1.0, 0.1],  
                      [1.0, 1.0, 1.0],  
                      [5.0, 1.0, 1.0]])  
    probs = softmax(logits)  
    for row_logits, row_probs in zip(logits, probs):  
        print(f"logits={row_logits} -> probs={np.round(row_probs, 4)} " "  
              f"(sum={row_probs.sum():.6f})")
```

Three hand-picked example rows, chosen to show three different behaviors: a mild gap, a perfect tie, and a large gap. **zip(logits, probs)** walks both 2D arrays row by row in lockstep since **softmax** preserved the same (3, 3) shape as the input. **np.round(row_probs, 4)** rounds only for display — the underlying array keeps full floating-point precision.

SYSTEM DIAGRAM

Softmax + cross-entropy pipeline (Segments 1-2)



The full pipeline this and the next segment build: logits become probabilities via softmax, which are then compared against the true class (one-hot encoded) to produce a single loss number.

Segment 2 — One-Hot Encoding and Categorical Cross-Entropy (lines 72–102)

MATH

$$\text{one_hot}(y)[i, j] = 1 \text{ if } j == y[i] \text{ else } 0$$

```
def one_hot(y, num_classes):
    m = y.shape[0]
    encoded = np.zeros((m, num_classes))
    encoded[np.arange(m), y] = 1.0
    return encoded
```

`np.zeros((m, num_classes))` allocates an all-zero (m, K) array. `encoded[np.arange(m), y] = 1.0` is **fancy indexing**: `np.arange(m)` is `[0, 1, 2, ..., m-1]` (the row indices) and `y` is the array of true class labels (the column indices) — pairing them up elementwise sets `encoded[0, y[0]]`, `encoded[1, y[1]]`, and so on to 1.0 in a **single vectorized line**, with no explicit loop over rows.

MATH

$$CE(\text{probs}, y_{\text{onehot}}) = -\text{mean}[\text{sum}(y_{\text{onehot}} \times \log(\text{probs}), \text{per row})]$$

(probs clipped to [eps, 1-eps] first so log(0) never happens)

```
def cross_entropy_loss(probs, y_onehot, eps=1e-9):
    probs = np.clip(probs, eps, 1 - eps)
    return -np.mean(np.sum(y_onehot * np.log(probs), axis=1))
```

The same `np.clip` numerical-safety pattern from every prior day's loss function. `y_onehot * log(probs)` multiplies elementwise — since `y_onehot` is zero everywhere except the true class, this zeroes out every column's log-probability except the correct one. `np.sum(..., axis=1)` collapses each row down to that single surviving value (per example), and the outer `np.mean(...)` averages across all examples in the batch.

```
def demo_cross_entropy():
    y = np.array([0, 2, 1])
    y_onehot = one_hot(y, num_classes=3)
    print(f"y = {y}")
    print(f"one_hot(y) = \n{y_onehot}")
```

Three example labels, deliberately one of each class (0, 2, 1) so the printed `one_hot(y)` array visibly shows a 1 landing in a different column on every row. `\n` inside the f-string inserts a literal newline so the printed array starts on its own line rather than trailing the label text.

```

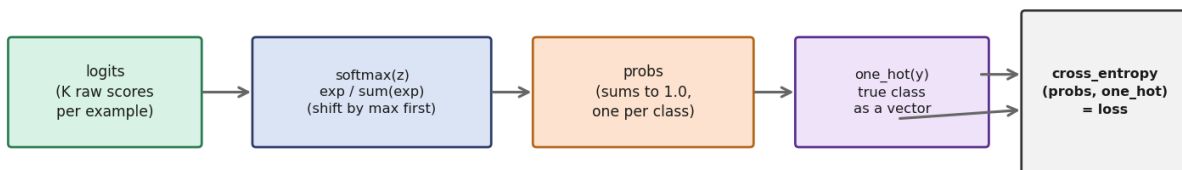
probs_good = np.array([[0.9, 0.05, 0.05],
                       [0.05, 0.05, 0.9],
                       [0.1, 0.8, 0.1]])
probs_bad = np.array([[0.2, 0.4, 0.4],
                      [0.4, 0.4, 0.2],
                      [0.4, 0.2, 0.4]])
ce_good = cross_entropy_loss(probs_good, y_onehot)
ce_bad = cross_entropy_loss(probs_bad, y_onehot)

```

probs_good assigns the largest probability in each row to the TRUE class from **y** above (row 0's largest is column 0, matching $y[0]=0$, and so on) — confidently correct. **probs_bad** also technically favors the correct class in each row, but only barely, with probability mass spread almost evenly across all three columns — unsure rather than wrong.

SYSTEM DIAGRAM

Softmax + cross-entropy pipeline (Segments 1-2)



The same pipeline diagram from Segment 1 — this segment builds its final two boxes: one-hot encoding the true label, then combining it with softmax's probabilities to compute cross-entropy.

Segment 3 — The Multi-Class Dataset (lines 109–120)

```
def make_multiclass_dataset():
    X, y = make_blobs(n_samples=300, centers=3, n_features=2,
                      cluster_std=1.8, random_state=42)
    return X, y
```

make_blobs (a new import this lesson, replacing prior days' **make_moons**) generates points from **centers=3** separate Gaussian clusters — **n_features=2** keeps each point 2D for easy plotting, and **cluster_std=1.8** controls how spread out (and therefore how overlapping) each cluster is.

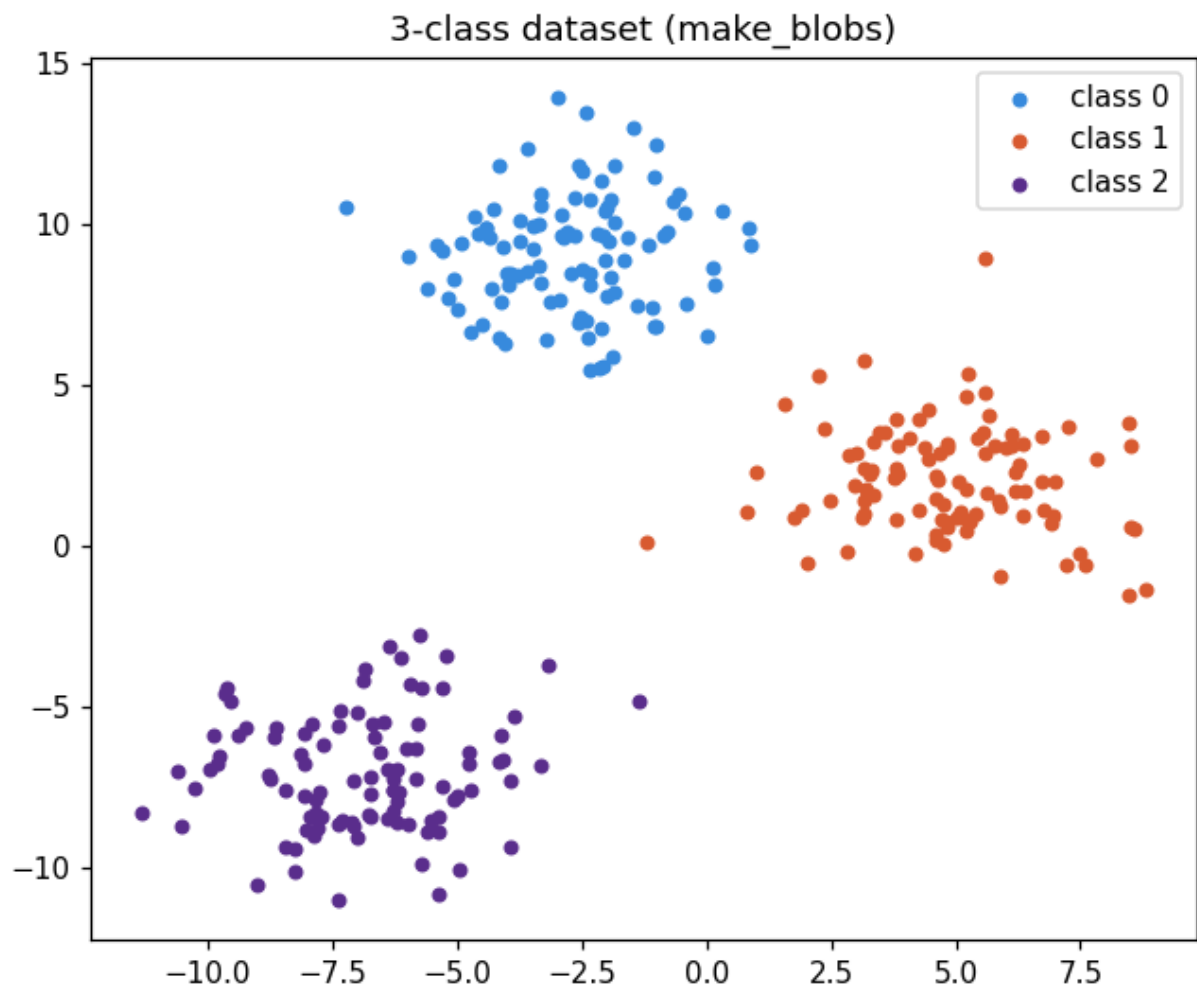
```
def demo_dataset():
    X, y = make_multiclass_dataset()
    classes = sorted(set(y.tolist()))
    print(f"X shape: {X.shape}   y shape: {y.shape}")
    print(f"classes: {classes}")
    for c in classes:
        print(f"   class {c}: {(y == c).sum()} points")
```

set(y.tolist()) converts the label array to a plain Python list, then to a **set** to get each UNIQUE class value with duplicates removed; **sorted(...)** orders them for a stable, readable printout (0, 1, 2 rather than whatever order a set happens to iterate in). **(y == c).sum()** is the same boolean-array-sum trick from every prior day: a boolean array's **True** values count as 1 when summed.

```
fig, ax = plt.subplots(figsize=(6, 5))
colors_map = {0: "#378ADD", 1: "#D85A30", 2: "#5a2d8c"}
for c in sorted(set(y.tolist())):
    mask = y == c
    ax.scatter(X[mask, 0], X[mask, 1], s=18,
               color=colors_map[c], label=f"class {c}")
ax.legend()
```

colors_map is a plain dictionary mapping each class integer to a fixed hex color, so every plot in this lesson uses the same color per class consistently. **mask = y == c** builds a boolean array (True where this row belongs to class **c**); **X[mask, 0]** and **X[mask, 1]** use that mask to pull out just this class's x- and y-coordinates for one **ax.scatter** call per class — the same boolean-mask-indexing idea used throughout this roadmap, applied here to select rows for plotting instead of for a loss computation.

REAL GRAPH (script's own output)



Not a mockup — this is `multiclass_dataset.png`, this exact script rerun fresh: 100 points per class, plotted with the loop above.

Segment 4 — SoftmaxNet.__init__ (lines 128–140)

```
class SoftmaxNet:
    """2-layer network: sigmoid hidden layer, softmax output layer over K
    classes. ..."""

    def __init__(self, n_in, n_hidden, n_classes, lr=0.5, seed=42):
        rng = np.random.RandomState(seed)
        self.W1 = rng.randn(n_in, n_hidden) * 0.5
        self.b1 = np.zeros(n_hidden)
        self.W2 = rng.randn(n_hidden, n_classes) * 0.5
        self.b2 = np.zeros(n_classes)
        self.lr = lr
```

The same weight-initialization pattern as every prior day's network, with one structural difference: **self.W2** is shaped **(n_hidden, n_classes)** instead of **(n_hidden, 1)** — the ONLY shape change needed anywhere in this class to go from binary to multi-class output. **n_classes** is a constructor argument (not hard-coded to 3), so this exact same class trains a classifier with any number of classes.

Segment 5 — SoftmaxNet.forward (lines 142–147)

MATH

$$z1 = X \times W1 + b1 \quad a1 = \text{sigmoid}(z1)$$
$$z2 = a1 \times W2 + b2 \quad \text{probs} = \text{softmax}(z2)$$

```
def forward(self, X):
    self.z1 = X @ self.W1 + self.b1
    self.a1 = sigmoid(self.z1)
    self.z2 = self.a1 @ self.W2 + self.b2
    self.probs = softmax(self.z2)
    return self.probs
```

Identical structure to every prior day's forward pass through the hidden layer. The only change is the last two lines: **self.z2** is now (m, K) instead of (m, 1), and the final activation is **softmax(self.z2)** (Segment 1) instead of **sigmoid(self.z2)** — stored as **self.probs** rather than **self.a2**, a naming choice reflecting that it's now a full probability distribution per example, not a single number.

Segment 6 — SoftmaxNet.backward (lines 149–160)

MATH

$dz2 = probs - y_onehot$ (the softmax+cross-entropy simplification)
 $dW2 = (1/m) \times a1^T \times dz2$ $db2 = mean(dz2)$
 $da1 = dz2 \times W2^T$ $dz1 = da1 \times a1 \times (1-a1)$
 $dW1 = (1/m) \times X^T \times dz1$ $db1 = mean(dz1)$

```
def backward(self, X, y_onehot):
    m = X.shape[0]
    dz2 = self.probs - y_onehot
    dW2 = self.a1.T @ dz2 / m
    db2 = dz2.mean(axis=0)
    da1 = dz2 @ self.W2.T
    dz1 = da1 * self.a1 * (1 - self.a1)
    dW1 = X.T @ dz1 / m
    db1 = dz1.mean(axis=0)
```

dz2 = self.probs - y_onehot is the entire payoff of Segment 1-2's math: softmax combined with categorical cross-entropy produces this single clean subtraction as the output-layer gradient, with no explicit softmax-derivative or log-derivative terms appearing anywhere in the code. Every line after it — **dW2**, **db2**, **da1**, **dz1**, **dW1**, **db1** — is CHARACTER-FOR-CHARACTER identical to Day 37's binary-classification backward pass; only the shape of the arrays flowing through changed (K columns instead of 1), not a single formula.

MATH

$param -= lr \times gradient$ (for each of W1, b1, W2, b2)

```
self.W1 -= self.lr * dW1
self.b1 -= self.lr * db1
self.W2 -= self.lr * dW2
self.b2 -= self.lr * db2
```

The same plain gradient-descent update seen every prior day — no optimizer pluggability or regularization mixed in here, keeping this lesson's comparison isolated to softmax/batching alone.

Segment 7 — SoftmaxNet.train (lines 162–182)

```
def train(self, X, y, epochs, batch_size=None, seed=0):
    num_classes = self.W2.shape[1]
    y_onehot_full = one_hot(y, num_classes)
    m = X.shape[0]
    rng = np.random.RandomState(seed)
    losses = []
    updates_per_epoch = 1 if batch_size is None else int(np.ceil(m / batch_size))
```

self.W2.shape[1] reads the number of classes directly off the already-built weight matrix rather than requiring it as a separate argument. **y_onehot_full** is computed ONCE, outside the epoch loop — since the true labels never change during training, there's no need to recompute their one-hot encoding on every epoch.

updates_per_epoch is a conditional expression computing, for logging purposes only, how many weight updates this call will perform per epoch — 1 for full-batch, or the number of chunks (**ceil(m / batch_size)**) for mini-batch.

```
    for epoch in range(epochs):
        if batch_size is None:
            self.forward(X)
            self.backward(X, y_onehot_full)
```

The full-batch branch: exactly one **forward/backward** call per epoch, using the ENTIRE dataset **X** and **y_onehot_full** — identical to every prior day's training loop.

MATH

idx = random shuffle of [0, 1, ..., m-1] (new shuffle every epoch)
for each chunk of batch_size indices: forward+backward on just that chunk

```
    else:
        idx = rng.permutation(m)
        for start in range(0, m, batch_size):
            batch_idx = idx[start:start + batch_size]
            self.forward(X[batch_idx])
            self.backward(X[batch_idx], y_onehot_full[batch_idx])
```

rng.permutation(m) returns a freshly shuffled array of **[0, 1, ..., m-1]** — a DIFFERENT random order every single epoch, since this line sits inside the **for epoch** loop. **range(0, m, batch_size)** steps through in chunks of **batch_size**; **idx[start:start + batch_size]** slices out one chunk of (already shuffled) row indices. **X[batch_idx]** and **y_onehot_full[batch_idx]** use FANCY INDEXING (Segment 2) to pull out just those rows — the same indexing style, just used here to select a mini-batch instead of writing a one-hot vector.

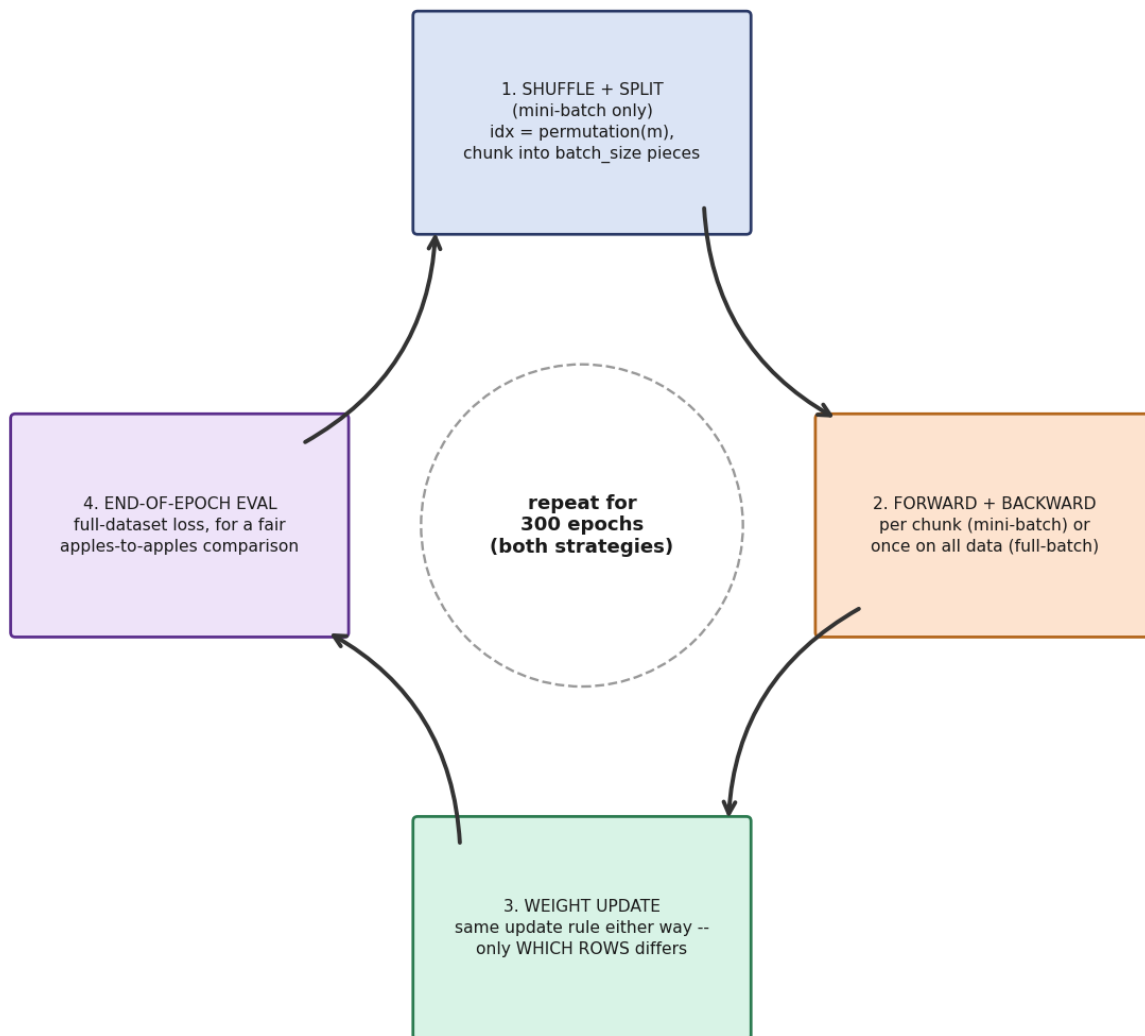
```
        probs_full = self.forward(X)
        losses.append(cross_entropy_loss(probs_full, y_onehot_full))

    return losses, updates_per_epoch
```

Regardless of which branch ran above, EVERY epoch ends with one more forward pass on the FULL dataset, purely to measure and record loss — this is what makes full-batch and mini-batch runs directly comparable epoch-for-epoch later, even though mini-batch's actual training happened on smaller chunks.

SYSTEM DIAGRAM

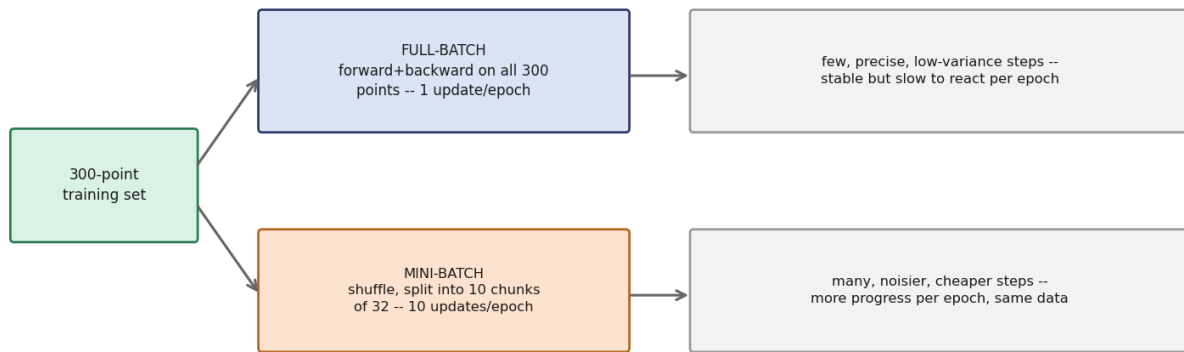
The training work cycle, extended for mini-batching (train())



This segment's loop as a work cycle. Stage 1 (shuffle+split) only happens for mini-batch; stage 2 runs once per chunk (mini-batch) or once total (full-batch); stage 4's end-of-epoch full-dataset evaluation runs identically either way.

SYSTEM DIAGRAM

Full-batch vs mini-batch: two update strategies, same data (Segments 5-6)



Zooming out on the choice this segment's `batch_size` argument controls: same 300-point dataset, fundamentally different number of weight updates per epoch.

Segment 8 — SoftmaxNet.accuracy (lines 184–187)

MATH

accuracy = mean([argmax(probs, per row) == y])

```
def accuracy(self, X, y):  
    probs = self.forward(X)  
    pred = probs.argmax(axis=1)  
    return (pred == y).mean()
```

probs.argmax(axis=1) is the multi-class equivalent of every prior day's **>= 0.5** threshold: for each row, it returns the COLUMN INDEX of the largest probability — i.e., which class the network is most confident about. Comparing directly against the integer labels **y** (no one-hot encoding needed here) and averaging the matches gives accuracy, exactly as in every prior day.

Segment 9 — demo_full_batch and demo_mini_batch (lines 195–222)

```
def demo_full_batch(X, y):  
    net = SoftmaxNet(n_in=2, n_hidden=16, n_classes=3, lr=0.5, seed=42)  
    losses, updates_per_epoch = net.train(X, y, epochs=300, batch_size=None)
```

batch_size=None explicitly requests the full-batch branch from Segment 7. **seed=42** on the network's constructor guarantees this network starts from the IDENTICAL initial weights as the mini-batch network below — the only fair way to isolate batching as the single variable being compared.

```
def demo_mini_batch(X, y):  
    net = SoftmaxNet(n_in=2, n_hidden=16, n_classes=3, lr=0.5, seed=42)  
    losses, updates_per_epoch = net.train(  
        X, y, epochs=300, batch_size=32, seed=0)
```

Same architecture, same **seed=42** for weight initialization, same **epochs=300** budget as **demo_full_batch** above — the ONLY differences are **batch_size=32** (triggering Segment 7's mini-batch branch) and the separate **seed=0**, which controls only the SHUFFLE order inside **train()**, not the weights.

```
print(f"Epoch 0:    loss={losses[0]:.4f}")  
print(f"Epoch 50:   loss={losses[50]:.4f}")  
print(f"Epoch 299:  loss={losses[-1]:.4f}")  
print(f"Final accuracy: {net.accuracy(X, y):.3f}")
```

The same specific-epoch f-string indexing pattern from Day 38: **losses[0]**, **losses[50]**, and **losses[-1]** (negative indexing for the last epoch) print a consistent snapshot for both demos, making the two printed blocks directly comparable line by line.

Segment 10 — demo_comparison and note_on_pytorch (lines 224–252)

```
def demo_comparison(fb_losses, fb_upe, mb_losses, mb_upe):  
    fig, ax = plt.subplots(figsize=(8, 5))  
    fb_label = f"full-batch ({fb_upe} update/epoch)"  
    mb_label = f"mini-batch ({mb_upe} updates/epoch)"  
    ax.plot(fb_losses, label=fb_label, color="#378ADD", linewidth=1.5)  
    ax.plot(mb_losses, label=mb_label, color="#D85A30", linewidth=1.5)
```

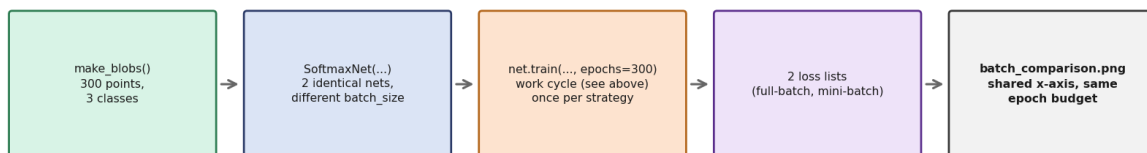
Both loss lists plotted on ONE shared axis (unlike Day 38's 2x2 grid) — appropriate here since there are only two configurations to compare, not four. The f-string labels bake the actual **updates_per_epoch** number directly into the legend, so the chart is self-documenting about exactly how many updates each line represents.

```
plt.savefig("batch_comparison.png", dpi=110)
```

Saves the completed comparison chart straight to a PNG file — this exact call is what produced the real graph on the next page.

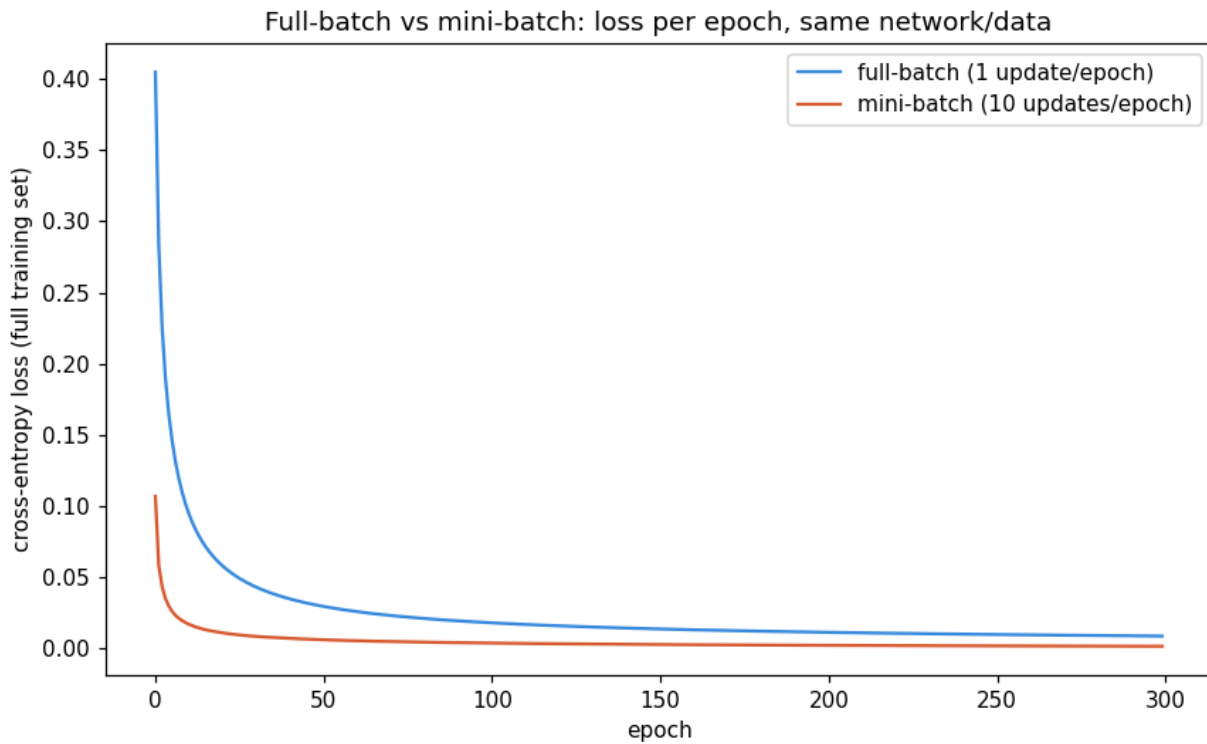
SYSTEM DIAGRAM

Segment 6 pipeline: how code + math becomes the final comparison chart



Zooming out: the full pipeline this segment runs end to end, from the shared dataset through two independent training runs to the final comparison chart — every box here is code you've now read line by line.

REAL GRAPH (script's own output)



Not a mockup — this is `batch_comparison.png`, this exact script rerun fresh. Mini-batch (orange) drops faster and settles lower than full-batch (blue) within the identical 300-epoch budget.

```
nn.CrossEntropyLoss()  
# takes RAW logits, not softmax output --  
# it applies log-softmax internally  
  
DataLoader(dataset, batch_size=32, shuffle=True)
```

note_on_pytorch() is just **print()** calls containing plain text — not executable code, and PyTorch is never imported anywhere in this file. These are reference lines mapping each hand-written mechanism to its PyTorch equivalent, with an explicit reminder in the comment that **nn.CrossEntropyLoss** expects raw logits, not already-softmaxed probabilities.

Segment 11 — main() and the Entry-Point Guard (lines 254–268)

```
def main():
    demo_softmax()
    demo_cross_entropy()
    X, y = demo_dataset()
    fb_losses, fb_upe = demo_full_batch(X, y)
    mb_losses, mb_upe = demo_mini_batch(X, y)
    demo_comparison(fb_losses, fb_upe, mb_losses, mb_upe)
    note_on_pytorch()
```

Same pattern as every prior day: one function calling every demo in the exact order they should run, with the SAME dataset (**X, y** from **demo_dataset()**) explicitly threaded into both training demos — making the shared-data requirement for a fair comparison visible directly in this function's structure.

```
if __name__ == "__main__":
    main()
```

The same entry-point guard as every script in this roadmap: **__name__** only equals "**__main__**" when this file is run directly, so **main()** won't silently re-execute if **SoftmaxNet** or **softmax** is ever imported from this file elsewhere.

Quick Syntax Glossary — New or Reinforced Constructs

`array.max(axis=1, keepdims=True)` — Reduces across columns (`axis=1`), computing one value per row. `keepdims=True` preserves the 2D shape so the result can broadcast cleanly against the original array in a later operation.

`array[np.arange(m), y] = value` — Fancy indexing: pairs up a row-index array with a column-index array elementwise, letting a single line set one specific cell per row without an explicit loop.

`set(list)` then `sorted(...)` — Converts a list to a set to get unique values (removing duplicates), then sorts them for stable, readable, repeatable ordering.

`dict` as a lookup table (`colors_map`) — A plain `{key: value}` dictionary used to consistently map a category (like a class label) to a fixed display property (like a color) across multiple plot calls.

boolean mask indexing for plotting — `mask = (y == c)` followed by `X[mask, 0]` selects just the rows belonging to one category — the same technique used elsewhere in this roadmap for filtering, applied here to split data for separate scatter-plot calls.

`value_if_true if condition else value_if_false` — A conditional (ternary) expression on one line — used here to compute `updates_per_epoch` differently depending on whether `batch_size` was given.

`rng.permutation(m)` — Returns a freshly shuffled array of integers `0..m-1` — calling it again produces a DIFFERENT random order, which is exactly why it belongs inside the epoch loop rather than before it.

`range(0, m, batch_size)` — The three-argument form of `range`: start, stop, and step — steps through in fixed-size jumps, used here to walk through a shuffled index array one chunk at a time.

`probs.argmax(axis=1)` — Returns the COLUMN INDEX of the largest value in each row — the multi-class generalization of a `>= 0.5` threshold, since there's no single fixed cutoff when there are more than two classes.

`np.ceil(m / batch_size)` — Rounds up to the nearest whole number — used here so a dataset that doesn't divide evenly by `batch_size` still counts its final, smaller chunk as one additional update.

End of Day 39 syntax reference. For the softmax, cross-entropy, and mini-batch concepts these lines implement, see the main Day 39 study guide PDF.